



## **RELATÓRIO FINAL: BAITA EVENTOS**

### **ENGENHARIA DE SOFTWARE SEGURO**

**DOCENTE: SILVIO QUINCONZES**

**DISCENTES: ANDRESSA ASSAE, HANNA MENDONÇA, MAIARA NERI,  
MIRIELI OLIVEIRA, NATALIA PERRI E VITORIA GARCIA**

**AGOSTO/2026**

## SUMÁRIO

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>1. INTRODUÇÃO.....</b>                                                | <b>3</b>  |
| <b>2. ENTREGA 1 - MODELAGEM DE AMEAÇAS (STRIDE) E CASOS DE ABUSO..</b>   | <b>4</b>  |
| 2.1 Descrição do sistema.....                                            | 4         |
| 2.2 Usuários, ativos e pontos de interação.....                          | 4         |
| 2.3 Arquitetura.....                                                     | 4         |
| 2.4 Modelagem STRIDE.....                                                | 4         |
| 2.5 Casos de abuso.....                                                  | 5         |
| 2.6 Síntese e considerações.....                                         | 5         |
| <b>3. ENTREGA 2 - ANÁLISE E TRATAMENTO DE RISCOS (NIST CSF 2.0).....</b> | <b>6</b>  |
| 3.1 Critérios e registro de riscos.....                                  | 6         |
| 3.2 Priorização.....                                                     | 6         |
| 3.3 Estratégias e funções do NIST CSF 2.0.....                           | 7         |
| 3.4 Mapeamento NIST.....                                                 | 7         |
| 3.5 Plano de tratamento.....                                             | 7         |
| 3.6 Implementação e risco residual.....                                  | 7         |
| <b>4. ENTREGA 3 - ARQUITETURA SEGURA.....</b>                            | <b>8</b>  |
| 4.1 Requisitos de segurança.....                                         | 8         |
| 4.2 Vulnerabilidades catalogadas.....                                    | 8         |
| 4.3 Decisões de arquitetura e rastreabilidade.....                       | 8         |
| <b>5. ENTREGA 4 - CÓDIGO SEGURO E TESTES.....</b>                        | <b>10</b> |
| 5.1 Prática 1 - Controle de autorização na API REST (R02 / RS03).....    | 10        |
| 5.2 Prática 2 - Proteção de sessões autenticadas (R01 / RS02).....       | 10        |
| <b>6. ENTREGA 5 - VERIFICAÇÃO DE VULNERABILIDADES.....</b>               | <b>11</b> |
| <b>7. ENTREGA 6 - MONITORAMENTO E DETECÇÃO DE INTRUSÕES.....</b>         | <b>12</b> |
| <b>8. ENTREGA 7 - VÍDEO FINAL.....</b>                                   | <b>13</b> |
| <b>9. CONSIDERAÇÕES FINAIS DO PROJETO.....</b>                           | <b>13</b> |
| <b>10. ESTRUTURA DO REPOSITÓRIO.....</b>                                 | <b>14</b> |

# 1. INTRODUÇÃO

Este relatório consolida o trabalho desenvolvido ao longo da disciplina de Engenharia de Software Seguro, aplicado ao sistema **BAITA EVENTOS**, uma plataforma para gestão de eventos acadêmicos, institucionais, culturais e corporativos, que centraliza a criação de eventos, a configuração de atividades, a inscrição de participantes, o controle de vagas, o check-in, a avaliação de atividades e o acompanhamento de resultados.

O trabalho foi organizado em sete entregas incrementais, cada uma correspondendo a uma etapa do ciclo de segurança do software: modelagem de ameaças, análise de riscos, arquitetura segura, código seguro, verificação de vulnerabilidades, detecção de intrusões e, por fim, um vídeo de encerramento apresentando os resultados. Cada etapa manteve rastreabilidade com a anterior, de modo que ameaças identificadas na Etapa 1 originaram os riscos priorizados na Etapa 2, que por sua vez originaram os requisitos e decisões de arquitetura da Etapa 3, as práticas de código seguro da Etapa 4 e as regras de detecção da Etapa 6.

A seguir, cada entrega é resumida com seus principais artefatos, decisões e resultados.

## **2. ENTREGA 1 - MODELAGEM DE AMEAÇAS (STRIDE) E CASOS DE ABUSO**

### **2.1 Descrição do sistema**

O BAITA EVENTOS resolve o problema da dispersão de ferramentas (planilhas, formulários, aplicativos de mensagens) na organização de eventos, centralizando inscrições, controle de vagas, check-in, avaliações e auditoria em uma única plataforma. O sistema possui quatro perfis de usuário: participante (sem conta, acessa a página pública), organizador, avaliador e administrador (usuários internos autenticados), e protege recursos como contas e credenciais, dados pessoais dos participantes, inscrições, registros de presença, avaliações, configurações de eventos, permissões e o banco de dados.

### **2.2 Usuários, ativos e pontos de interação**

Foram detalhados os quatro perfis de acesso e onze ativos do sistema (contas e credenciais, papéis e permissões, solicitações de associação de organizadores, dados de inscrição, inscrições, registros de check-in, avaliações e resultados, configurações, registros de auditoria, banco de dados e disponibilidade da plataforma), além de oito pontos de interação (página pública, tela de autenticação, área autenticada, fluxo de aprovação de organizadores, API REST, banco de dados, códigos de check-in e serviço de e-mail).

### **2.3 Arquitetura**

A arquitetura adotada é cliente-servidor em três camadas: apresentação (aplicação web em React, dividida em página pública por evento e área autenticada), aplicação (API REST em Node.js/Express, responsável por autenticação, autorização e regras de negócio) e dados (MongoDB, acessado exclusivamente pela API). Foram descritos os fluxos principais: inscrição, check-in e avaliação, e as fronteiras de confiança entre navegador, aplicação web, API, banco de dados e serviço de e-mail.

### **2.4 Modelagem STRIDE**

A modelagem foi realizada com o Microsoft Threat Modeling Tool, a partir de um diagrama de fluxo de dados (DFD) que representa os quatro perfis de usuário, a página pública, a área autenticada, a API REST, o MongoDB e o serviço de e-mail externo, conectados pelas respectivas fronteiras de confiança.

A ferramenta gerou 111 ameaças automáticas, das quais 30 foram selecionadas

após revisão manual do grupo, contextualizadas para a arquitetura do BAITA Eventos e distribuídas entre as seis categorias STRIDE:

| <b>Categoria</b>       | <b>Quantidade</b> |
|------------------------|-------------------|
| Spoofing               | 7                 |
| Tampering              | 3                 |
| Repudiation            | 6                 |
| Information Disclosure | 2                 |
| Denial of Service      | 6                 |
| Elevation of Privilege | 6                 |
| <b>Total</b>           | <b>30</b>         |

tabela 1

Cada ameaça mantida recebeu um identificador T seguido do ID numérico original da ferramenta, preservando a rastreabilidade.

## **2.5 Casos de abuso**

A partir das 30 ameaças selecionadas, foram elaborados 10 casos de abuso (CA01–CA10), agrupando ameaças relacionadas em cenários concretos de uso malicioso da plataforma — desde o comprometimento de contas internas até a indisponibilidade provocada por sobrecarga das rotas públicas e da API.

## **2.6 Síntese e considerações**

As ameaças e casos de abuso mais preocupantes identificados foram: comprometimento da conta de administrador (Spoofing), falhas de autorização entre perfis (Elevation of Privilege), acesso indevido a dados pessoais das inscrições (Information Disclosure), alteração de avaliações sem rastreabilidade (Tampering/Repudiation), execução de código indevido na API (Elevation of Privilege) e indisponibilidade em períodos críticos (Denial of Service). Os ativos mais críticos apontados foram as contas de usuários internos, os dados pessoais das inscrições, as avaliações/resultados, os registros de auditoria e o banco de dados/disponibilidade da plataforma. Essas conclusões formaram a base direta para a Etapa 2.

### 3. ENTREGA 2 - ANÁLISE E TRATAMENTO DE RISCOS (NIST CSF 2.0)

#### 3.1 Critérios e registro de riscos

Foram definidas escalas de probabilidade (1 a 4) e impacto (1 a 4), com pontuação calculada por Probabilidade × Impacto e classificação em Baixo (1–3), Médio (4–7), Alto (8–11) e Crítico (12–16). A partir dos 10 casos de abuso da Etapa 1, foram derivados 10 riscos (R01–R10), cada um mantendo a rastreabilidade com as ameaças STRIDE de origem:

| Risco | Descrição resumida                            | Pontuação | Classificação |
|-------|-----------------------------------------------|-----------|---------------|
| R01   | Comprometimento de contas autenticadas        | 12        | Crítico       |
| R02   | Execução de operações sem autorização         | 12        | Crítico       |
| R03   | Manipulação de avaliações                     | 9         | Alto          |
| R04   | Fraudes em inscrições e check-ins             | 12        | Crítico       |
| R05   | Inserção de conteúdo malicioso                | 12        | Crítico       |
| R06   | Operações indevidas via API REST              | 8         | Alto          |
| R07   | Comprometimento da comunicação API–MongoDB    | 8         | Alto          |
| R08   | Comprometimento de mensagens/tokens de e-mail | 8         | Alto          |
| R09   | Indisponibilidade da plataforma               | 6         | Crítico       |
| R10   | Execução de operações por CSRF                | 8         | Alto          |

tabela 2

#### 3.2 Priorização

A priorização considerou não apenas a pontuação, mas critérios complementares (ativos afetados, abrangência, recuperação, urgência, dependências e efeito acumulado). A ordem final foi: **R09 > R01 > R02 > R05 > R04 > R06 > R07 > R08 > R03 > R10**. O R09 (indisponibilidade) recebeu a primeira posição por afetar

simultaneamente todos os perfis de usuário e depender de componentes centrais (API REST e MongoDB).

### **3.3 Estratégias e funções do NIST CSF 2.0**

Foi adotada predominantemente a estratégia de *Reduzir*, já que a maioria dos riscos está ligada a funcionalidades essenciais que não podem ser eliminadas. A exceção foi o R08, tratado com a estratégia *Compartilhar*, dada a dependência do provedor externo de e-mail. As seis funções do NIST CSF 2.0 (Govern, Identify, Protect, Detect, Respond, Recover) foram utilizadas como estrutura de organização dos controles.

### **3.4 Mapeamento NIST**

Cada risco foi mapeado às funções pertinentes, com justificativa individual. *Identify*, *Protect*, *Detect* e *Respond* apareceram em todos os riscos; *Govern* foi associada a R01, R02, R06, R08 e R09 (riscos com componente de política/governança); *Recover* foi associada a R03, R05, R07 e R09 (riscos que podem exigir restauração de dados ou serviços).

### **3.5 Plano de tratamento**

Para cada risco foram definidos controles propostos, funções NIST relacionadas, responsáveis (equipe de desenvolvimento, administração da plataforma e/ou responsável pela infraestrutura) e evidências de verificação. Destacam-se: política de senhas e proteção de sessão (R01), controle de acesso baseado em perfis (R02), validação/sanitização de entradas (R05) e limitação de requisições com backups testados (R09).

### **3.6 Implementação e risco residual**

Foi definida uma ordem de implementação em 10 passos, priorizando infraestrutura de auditoria e monitoramento, seguida por rate limiting/backups, fortalecimento de autenticação, validação de entradas na API, proteção CSRF, reforço das inscrições públicas, segurança da comunicação com o MongoDB, revisão da integração de e-mail e, por fim, revisão integrada. As estimativas de risco residual indicaram redução geral das classificações, por exemplo, R09 (Crítico–16) para Médio–6, e R02 (Crítico– 12) para Médio–4, condicionadas à implementação e verificação efetiva dos controles propostos.

## 4. ENTREGA 3 - ARQUITETURA SEGURA

### 4.1 Requisitos de segurança

Os três riscos mais prioritários da Etapa 2 (R09, R01 e R02) originaram três requisitos de segurança verificáveis:

- **RS01 - Disponibilidade da plataforma** (origem: R09): limitação de requisições abusivas, registro de eventos de disponibilidade e cópias de segurança testadas do MongoDB.
- **RS02 - Proteção de contas autenticadas** (origem: R01): autenticação federada com Google via OpenID Connect (OIDC), com validação de tokens, sessões locais protegidas e invalidação em caso de comprometimento.
- **RS03 - Autorização de operações protegidas** (origem: R02): validação, no servidor, da autenticação, do perfil e do vínculo do usuário com o evento/atividade antes de cada operação protegida.

### 4.2 Vulnerabilidades catalogadas

Cada requisito foi relacionado a uma categoria CWE reconhecida:

| Risco | CWE                                                            | Requisito relacionado |
|-------|----------------------------------------------------------------|-----------------------|
| R09   | CWE-770 — Allocation of Resources Without Limits or Throttling | RS01                  |
| R01   | CWE-287 — Improper Authentication                              | RS02                  |
| R02   | CWE-862 — Missing Authorization                                | RS03                  |

tabela 3

### 4.3 Decisões de arquitetura e rastreabilidade

Foram formalizadas três decisões de arquitetura:

1. **Autenticação federada com Google (OIDC)** para organizadores, avaliadores e administradores, com validação de tokens e sessão local, trata R01/RS02.
2. **Validação de autorização na API REST**, considerando perfil e vínculo com o evento antes de cada operação, trata R02/RS03.
3. **Controles de disponibilidade e recuperação** (rate limiting, monitoramento, logs, backups do MongoDB e redundância quando viável) — trata R09/RS01.



O diagrama da arquitetura segura (figura 1) representa o fluxo Cliente/Página pública → Google Identity/API REST (rate limit, auth, logs) → Serviços de aplicação (regras e autorização) → Dados (MongoDB + backups)/Monitoramento (logs e alertas), e o diagrama de controles resume os quatro blocos: identidade e acesso, autorização na API, disponibilidade/recuperação e auditoria/sessões.

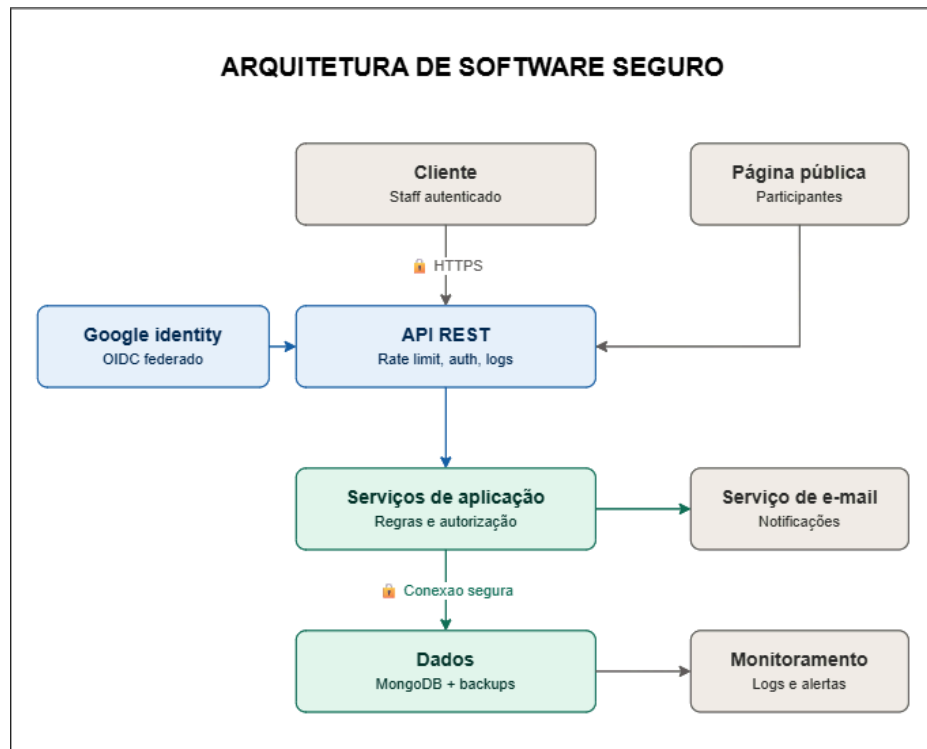


figura 1

## 5. ENTREGA 4 - CÓDIGO SEGURO E TESTES

### 5.1 Prática 1 - Controle de autorização na API REST (R02 / RS03)

- **TS01 - Operação autorizada:** organizador vinculado ao evento solicita atualização de atividade → solicitação permitida e registrada em log.
- **TS02 - Operação não autorizada:** organizador tenta atualizar atividade de outro evento → resposta 403 — Permissão insuficiente , sem alteração de dados, tentativa registrada.

A implementação em pseudocódigo ( atualizarAtividade ) valida, nesta ordem, a sessão, o perfil ( organizador ) e o vínculo com o evento, antes de aplicar qualquer alteração. Baseada na OWASP Authorization Cheat Sheet, que recomenda negação por padrão e validação em todas as requisições protegidas.

### 5.2 Prática 2 - Proteção de sessões autenticadas (R01 / RS02)

- **TS03 - Autenticação válida:** token Google/OIDC válido, não expirado e emitido para a plataforma → sessão local criada e evento registrado.
- **TS04 - Token inválido ou expirado:** token expirado, com assinatura inválida ou destinado a outra aplicação → resposta 401 - Token inválido ou expirado , sem criação de sessão, sem armazenar o token em log.

A implementação ( autenticarComGoogle ) valida assinatura, emissor, destinatário e expiração do token antes de criar uma sessão local com identificador aleatório e cookie configurado com HttpOnly , Secure e SameSite=Lax . Baseada na OWASP Session Management Cheat Sheet.

Ambas as práticas transformam os requisitos de segurança da Etapa 3 em comportamentos testáveis, reforçando que autenticação e autorização são controles distintos e que a validação deve ocorrer sempre no servidor.

## 6. ENTREGA 5 - VERIFICAÇÃO DE VULNERABILIDADES

A verificação foi realizada em ambiente autorizado e local, utilizando o OWASP ZAP 2.17.0 contra o OWASP Juice Shop (aplicação deliberadamente vulnerável, executada via Docker). O escopo foi limitado aos recursos acessíveis a partir da aplicação local, sem envolver sistemas de terceiros.

A varredura identificou 2 alertas de risco médio, 1 de risco baixo e 3 informativos, dos quais três foram analisados em profundidade:

| ID  | Achado                                                          | Risco (ZAP) | CWE     | CW Correção proposta                                                                                                              |
|-----|-----------------------------------------------------------------|-------------|---------|-----------------------------------------------------------------------------------------------------------------------------------|
| A01 | Content Security Policy (CSP) Header Not Set                    | Médio       | CWE 693 | Configurar cabeçalho Content-Security-Policy restritivo, testado gradualmente.                                                    |
| A02 | Cross-Domain Misconfiguration (Access-Control-Allow Origin: * ) | Médio       | CWE 264 | Substituir o curinga * por lista explícita de origens autorizadas; revisar uso de credenciais em CORS.                            |
| A03 | Timestamp Disclosure - Unix                                     | Baixo       | CWE 200 | Avaliar necessidade da data exposta; remover/generalizar quando não essencial, ou aceitar como risco baixo com revisão periódica. |

tabela 4

Os resultados foram documentados com evidências (relatório HTML e capturas de tela) e reconhecem explicitamente a limitação da sessão: os achados dizem respeito ao Juice Shop (aplicação de treinamento) e não devem ser atribuídos automaticamente ao BAITA Eventos. O objetivo da atividade foi exercitar o uso autorizado da ferramenta e relacionar os achados às práticas de segurança estudadas nas etapas anteriores (CSP e CORS reforçam, por exemplo, as mesmas preocupações tratadas em R05 e RS03).

## 7. ENTREGA 6 - MONITORAMENTO E DETECÇÃO DE INTRUSÕES

Esta etapa estabelece a diferença entre prevenção (impedir o abuso antes que produza efeito) e detecção (perceber que uma tentativa ocorreu, mesmo quando a prevenção falha), destacando seu caráter complementar.

Foram listadas oito categorias de eventos que devem ser registrados no BAITA Eventos, incluindo tentativas de autenticação, alterações em eventos/permissões, registros de avaliação, inscrições/check ins, requisições rejeitadas por falha de autorização, volume de requisições por conta/IP/rota, conteúdo enviado em campos livres e métricas de tempo de resposta/erro.

Com base nos riscos de maior prioridade da Etapa 2, foram definidas três regras de detecção:

| Regra   | Risco de origem                       | Condição de alerta                                                                | Resposta inicial                                                                                       |
|---------|---------------------------------------|-----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Regra 1 | R09 (indisponibilida de / CA09)       | >50 req/min do mesmo IP para a mesma rota, ou tempo de resposta 3x acima da média | Limitar requisições da origem, alertar a equipe de infraestrutura, verificar impacto no MongoDB/e-mail |
| Regra 2 | R01 (comprometimento de conta / CA01) | >5 logins malsucedidos em 5 min, ou login de IP/dispositivo nunca usado           | Bloquear tentativas para a conta, notificar o usuário, registrar para análise                          |
| Regra 3 | R05 (conteúdo malicioso / CA05)       | Padrões de script ( <script , javascript: , onerror= ) em campos livres           | Rejeitar a requisição, registrar origem e conteúdo, alertar se o padrão se repetir                     |

tabela 5

Por fim, foi definido um processo padrão de resposta pós-alerta: confirmar a ameaça, avaliar o impacto já causado, aplicar a resposta correspondente, comunicar a equipe técnica quando confirmado, e registrar o incidente para retroalimentar as regras de detecção.

## **8. ENTREGA 7 - VÍDEO FINAL**

Como encerramento do projeto, o grupo produziu um vídeo final apresentando o percurso completo da disciplina aplicado ao BAITA Eventos: Sobre o Projeto, Ameaça e Casos de Abuso, Riscos Prioritários, Arquitetura Segura, Práticas de Código Seguro, Verificação de Segurança, Detecção de Intrusões, Pipeline DEVSECOPS, Conclusões.

## **9. CONSIDERAÇÕES FINAIS DO PROJETO**

O desenvolvimento do Baita Eventos ao longo da disciplina demonstrou, na prática, como um processo estruturado de engenharia de software seguro conecta etapas que muitas vezes são tratadas de forma isolada: a modelagem de ameaças orientou a análise de riscos; os riscos priorizados originaram requisitos de segurança verificáveis; os requisitos guiaram decisões de arquitetura e práticas de código; e os mesmos riscos de maior prioridade (R01, R05 e R09) reapareceram como base para as regras de detecção de intrusões, fechando o ciclo entre prevenção e monitoramento.

Os riscos considerados mais críticos ao longo de todo o projeto foram consistentes: indisponibilidade da plataforma (R09), comprometimento de contas autenticadas (R01) e execução de operações sem autorização (R02). Esses três riscos concentraram a maior parte dos requisitos, decisões de arquitetura, práticas de código e regras de detecção desenvolvidas.

Entre as principais dificuldades relatadas pelo grupo estiveram o volume de ameaças geradas automaticamente pela ferramenta STRIDE (111 ameaças, reduzidas a 30 por revisão manual), a necessidade de diferenciar ameaças, casos de abuso, vulnerabilidades e eventos de risco mantendo a rastreabilidade entre eles, e o caráter estimado dos níveis de risco residual, já que os controles propostos ainda não foram implementados e testados em produção.

Como próximos passos, o grupo recomenda: implementar os controles seguindo a ordem definida na Etapa 2, produzir evidências reais de verificação, executar testes de segurança de ponta a ponta na aplicação (e não apenas em ambiente de treinamento) e revisar periodicamente o registro de riscos e as regras de detecção conforme a plataforma evoluir.

## 10. ESTRUTURA DO REPOSITÓRIO

```
├── README.md
├── docs/
│   ├── entrega-1/ # ETAPA 1 — STRIDE e casos de abuso
│   │   ├── 01-descricao-do-sistema.md
│   │   ├── 02-usuarios-ativos.md
│   │   ├── 03-arquitetura.md
│   │   ├── 04-modelagem-stride.md
│   │   ├── 05-casos-de-abuso.md
│   │   └── 06-consideracoes-finais.md
│   ├── entrega-2/ # ETAPA 2 — Análise e tratamento de riscos (NIST CSF)
│   │   ├── 01-criterios-e-registro-de-riscos.md
│   │   ├── 02-priorizacao-dos-riscos.md
│   │   ├── 03-estrategias-e-funcoes-nist.md
│   │   ├── 04-mapeamento-nist.md
│   │   ├── 05-plano-de-tratamento.md
│   │   └── 06-implementacao-e-risco-residual.md
│   ├── entrega-3/ # ETAPA 3 — Arquitetura segura
│   │   ├── 01-02-03-requisitos-de-seguranca.md
│   │   ├── 04-vulnerabilidades-catalogadas.md
│   │   └── 05-06-decisoes-e-rastreabilidade.md
│   ├── entrega-4/ # ETAPA 4 — Código seguro e testes
│   │   ├── 01-contexto-e-testes-autorizacao.md
│   │   ├── 02-03-implementacao-e-resultados-autorizacao.md
│   │   ├── 04-contexto-e-testes-sesoes.md
│   │   └── 05-06-implementacao-e-resultados-sesoes.md
│   ├── entrega-5/ # ETAPA 5 — Verificação de vulnerabilidades
│   │   └── 01-verificacao-de-vulnerabilidades.md
│   ├── entrega-7/ # ETAPA 7 — DevSecOps e vídeo final
│   │   └── 01-devsecops-e-videofinal.md
│   ├── diagramas/
│   │   ├── diagrama-stride.png
│   │   └── modelo-de-ameacas.tm7
│   ├── evidencias/
│   │   ├── etapa-5/
│   │   │   ├── 01-resumo-alertas-zap.png
│   │   │   ├── 02-alerta-csp-ausente.png
│   │   │   ├── 03-alerta-cors.png
│   │   │   ├── 04-alerta-timestamp.png
│   │   │   └── relatorio-zap-juice-shop.html
│   │   └── materiais-complementares/
│   │       ├── ameacas-stride-original.csv
│   │       ├── ameacas-stride-original.xlsx
│   │       └── ameacas-stride-selecionadas.xlsx
│   ├── relatorio/
│   │   ├── relatorio-baita-eventos.md
│   │   └── relatorio-baita-eventos.pdf
│   └── roteiros/
│       ├── entrega-6/etapa-6-deteccao-de-intrusoes.md
│       └── entrega-7/entrega-7-devsecops-videofinal.md
```